

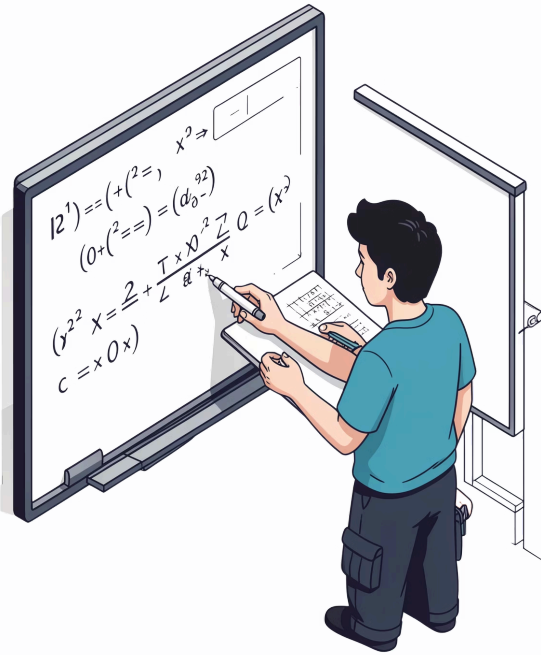


Prompt Engineering Techniques for Large Language Models

Zero-Shot · Few-Shot · Chain-of-Thought · Role Prompting · System Prompts ·
In-Context Learning

Dr. Anilkumar V. Brahmane

Department of Computer Engineering · Sanjivani College of Engineering



What is Prompt Engineering?

— Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering

Definition

The art and science of crafting inputs (prompts) to guide an LLM to produce the desired output

Why It Matters

Same model, different prompt → drastically different results — **no retraining needed**

Core Idea

LLMs are next-token predictors; a well-structured prompt steers the probability distribution toward correct answers

Analogy

Think of the LLM as a brilliant student — prompt engineering is *how you ask the exam question*

Overview: 6 Key Techniques at a Glance

1. Zero-Shot

Ask directly — no examples given

2. Few-Shot

Provide 2–5 examples before the real question

3. Chain-of-Thought

Ask the model to reason step-by-step

4. Role Prompting

Assign a persona or role to the model

5. System Prompts

Set global behavior rules before the conversation

6. In-Context Learning

Model learns from examples inside the prompt — no weight updates

TECHNIQUE 1

Zero-Shot Prompting

Concept

— Dr. Anilkumar V. Brahmane / Computer Engineering / Sanjivani College of Engineering



1 What It Is

Give the model a task with **NO examples** — just a direct instruction. The model relies entirely on its pre-trained knowledge.

2 Best For

Simple, well-defined tasks — classification, translation, summarization where the task definition is unambiguous.

3 Limitation

May fail on complex reasoning or domain-specific tasks without any guiding examples.

Zero-Shot: Python Code Example

Task: Classify the sentiment of a movie review – no examples provided to the model

```
import openai

client = openai.OpenAI(api_key="YOUR_API_KEY")

prompt = """Classify the sentiment of the following
review as Positive, Negative, or Neutral.

Review: "The movie had stunning visuals
        but a weak storyline."

Sentiment:"""

response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "user", "content": prompt}
    ]
)

print(response.choices[0].message.content)
```

✓ Expected Output

Neutral

The model correctly identifies the mixed review (positive visuals, negative plot) as Neutral – using zero-shot reasoning from pre-training alone.

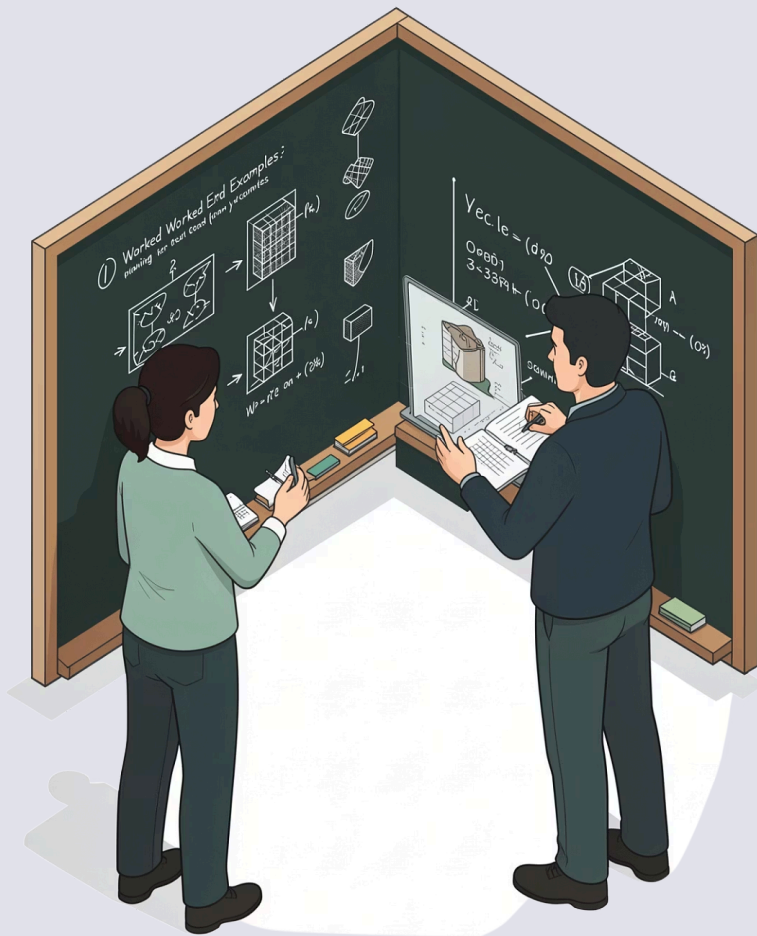
No Examples Needed

Model uses pre-trained knowledge of "Positive / Negative / Neutral" labels directly

Token Efficient

Minimal prompt → lowest API cost, fastest response time

Few-Shot Prompting — Concept



What It Is

Provide **2–5 labeled input-output examples** before the actual query

How It Works

Examples act as an **implicit pattern** — the model infers the task format and style from them

Key Insight

More examples → better accuracy, but too many → token limit issues.
Sweet spot: **3–5 examples**

Best For

Custom formats, domain-specific tasks, structured output generation, non-standard classifications

Few-Shot: Python Code Example

Task: Classify sentiment — this time with 3 labeled examples guiding the model

```
prompt = """Classify sentiment as  
Positive, Negative, or Neutral.
```

```
Review: "Absolutely loved it!"  
→ Positive
```

```
Review: "Terrible experience, never again."  
→ Negative
```

```
Review: "It was okay, nothing special."  
→ Neutral
```

```
Review: "Great acting but the  
plot was confusing."  
→ ""
```

```
response = client.chat.completions.create(  
    model="gpt-4",  
    messages=[  
        {"role": "user", "content": prompt}  
    ]  
)
```

```
print(response.choices[0].message.content)
```

✓ Expected Output

Neutral

The model learned the arrow (→) format AND the 3-class labeling scheme purely from the in-prompt examples.

Pattern Recognition

Model infers the "→ Label" output format from examples

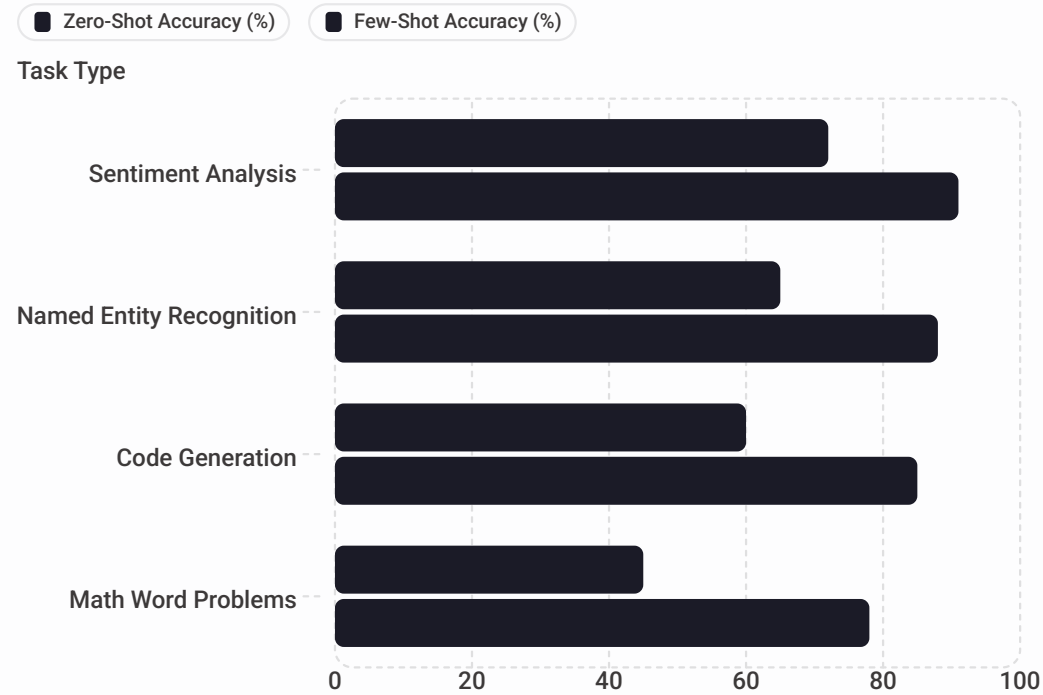
Context as Teacher

3 examples replace explicit task instructions entirely

No Retraining

Zero weight updates — all learning happens at inference time

Zero-Shot vs Few-Shot: Side-by-Side Comparison



When to Use Each

— Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering

Few-shot prompting consistently outperforms zero-shot on complex tasks across all categories.

Use Zero-Shot When

- General, well-known tasks
- Quick prototyping needed
- Token budget is tight

Use Few-Shot When

- Custom output format required
- Domain-specific classification
- Structured data extraction

Zero-Shot vs Few-Shot: Key Takeaways

+19%

Sentiment Gain

Few-shot improves accuracy from 72% → 91% on sentiment tasks

+33%

Math Word Problems

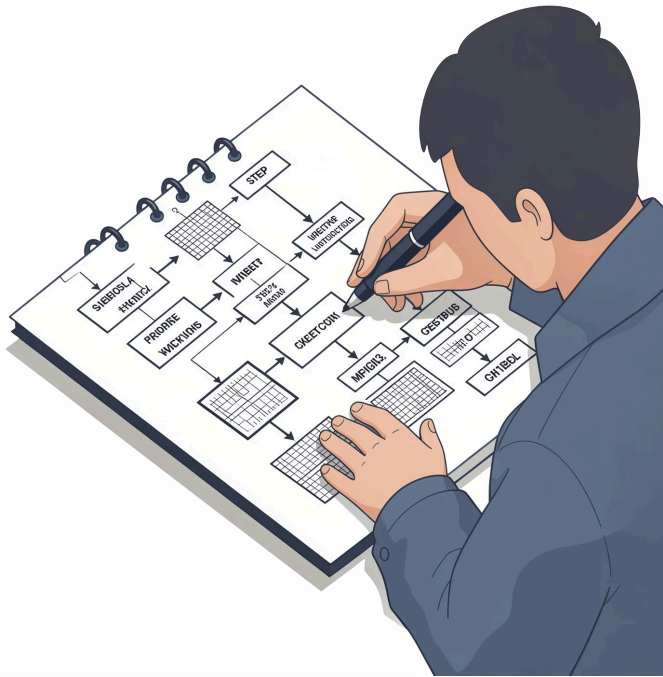
Largest gain: 45% → 78% — examples help most where reasoning is required

3–5

Optimal Examples

The sweet spot — enough to guide pattern, few enough to stay within token limits

❏ Few-Shot is the go-to upgrade when Zero-Shot accuracy is insufficient — always try it before reaching for fine-tuning.



TECHNIQUE 3

Chain-of-Thought (CoT) — Concept

— Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering

1 What It Is

Instruct the model to show its **reasoning step-by-step** before giving the final answer

3 Why It Works

Forces the model to **decompose** complex problems into smaller, verifiable sub-steps

2 Magic Phrase

"Let's think step by step" — dramatically improves accuracy on reasoning tasks

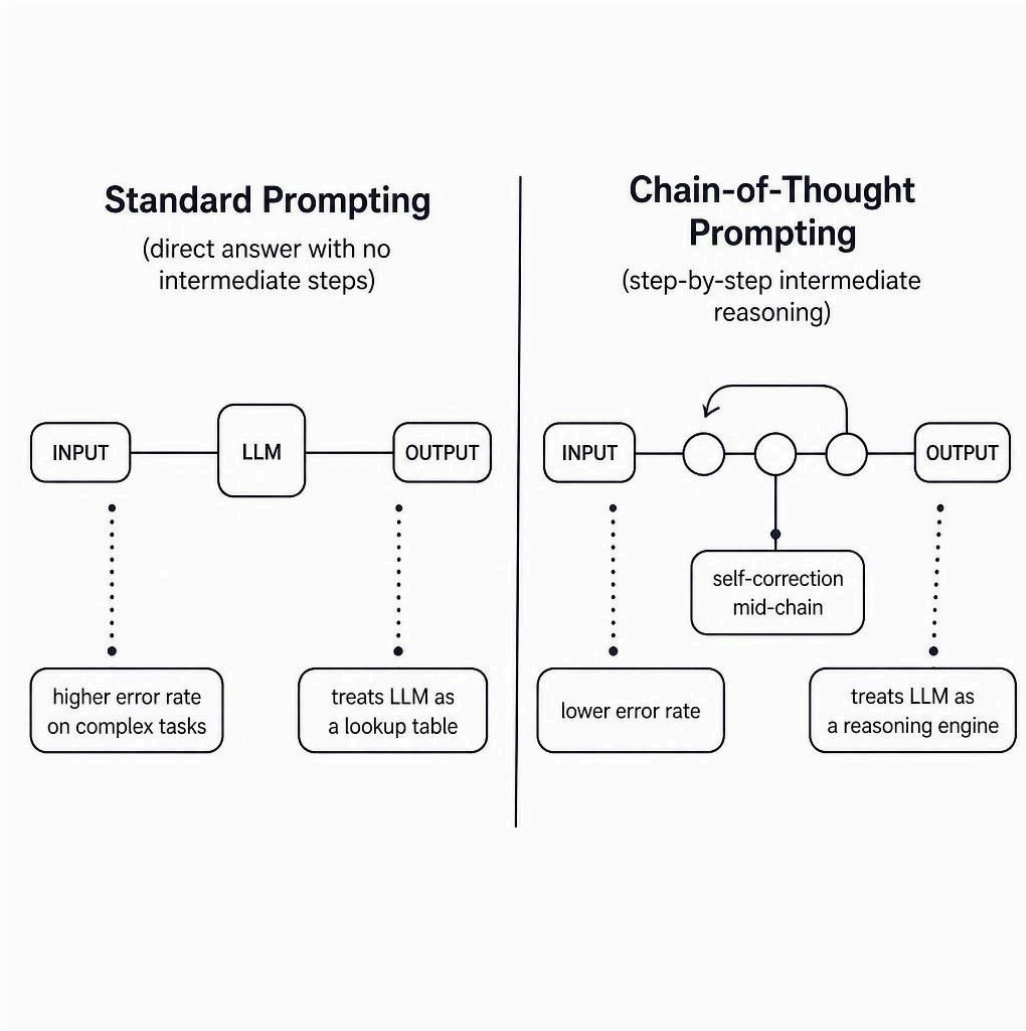
4 Best For

Math problems, logical reasoning, multi-step algorithms, debugging code

Chain-of-Thought: How It Works



Why Chain-of-Thought Works: The Science



Academic Insight

Wei et al. (2022) showed that CoT prompting emerges as a capability only in models with **100B+ parameters** – smaller models do not benefit.

– Dr. Anilkumar V. Brahmane / Computer Engineering / Sanjivani College of Engineering

Self-Correction

Intermediate steps allow the model to catch errors before committing to a final answer

Verifiability

Teachers and students can trace exactly where the model's reasoning went right or wrong

Generalization

Works across math, code debugging, logical puzzles, and multi-hop questions

Chain-of-Thought: Python Code Example

Task: Solve a math word problem using step-by-step reasoning triggered by "Let's think step by step"

```
prompt = """Solve the following problem  
step by step.
```

```
Problem: A train travels 60 km/h for  
2 hours, then 80 km/h for 3 hours.  
What is the total distance?
```

```
Let's think step by step:"""
```

```
response = client.chat.completions.create(  
    model="gpt-4",  
    messages=[  
        {"role": "user", "content": prompt}  
    ]  
)  
  
print(response.choices[0].message.content)
```

✓ Expected Output

Step 1: Distance for first leg
= $60 \text{ km/h} \times 2 \text{ h} = 120 \text{ km}$

Step 2: Distance for second leg
= $80 \text{ km/h} \times 3 \text{ h} = 240 \text{ km}$

Step 3: Total Distance
= $120 + 240 = \mathbf{360 \text{ km}}$

Key Trigger Phrase

"Let's think step by step"
— activates CoT
reasoning in GPT-4 class
models

Traceable Reasoning

Each step is visible and
verifiable — ideal for
academic and technical
use cases

Role Prompting — Concept



What It Is

Assign a specific **persona or expert role** to the model before asking your question

How It Works

The model adjusts **vocabulary, tone, depth, and perspective** to match the assigned role

Example Roles

"You are a senior Python developer" · "You are a cybersecurity expert" ·
"You are a professor teaching B.Tech students"

Best For

Technical explanations, code reviews, domain-specific advice, teaching content generation